

Videogame Semantic Search

Relazione Tecnica di Progetto

Ontologia OWL 2 per videogiochi: modellazione, reasoning e ricerca semantica

Videogame Semantic Search Project

25 settembre 2026

Versione 1.0

*Documento tecnico-accademico prodotto nell'ambito del progetto
di Semantic Web — Ontologia e Knowledge Graph per Videogiochi*

Sommario

Il presente documento descrive il progetto **Videogame Semantic Search**, un sistema di ricerca semantica che integra un'ontologia OWL 2 modellata sul dominio dei videogiochi, un knowledge graph popolato da Wikidata e un'interfaccia web con grafo interattivo. L'ontologia, costruita secondo i principi del Semantic Web, definisce 78 classi, 31 object property, 26 datatype property e 15 classi definite (più 2 classi vincolate da cardinalità massima) classificate tramite reasoning automatico. La popolazione da Wikidata (2010–2026) produce oltre 1.1 milioni di triple, interrogabili in linguaggio naturale tramite un agente SPARQL basato su GPT-4.1-mini. Il documento è centrato sull'ontologia: modellazione dello schema e scelte di design, espressività DL e profilo OWL 2, reasoning a due livelli (OWL 2 RL + CONSTRUCT) e interrogazione SPARQL della conoscenza inferita. Popolamento e applicazione web sono descritti in coda come contesto.

Indice

Indice	2
1 Introduzione	5
1.1 Contesto: Semantic Web e Ontologie	5
1.2 Obiettivo del Progetto	5
1.3 Casi d'Uso	5
1.4 Panoramica della Relazione	5
2 Panoramica dell'Ontologia	6
2.1 Dominio	6
2.2 Namespace e Separazione Schema/Istanze	6
2.3 Fonte Dati: Wikidata	6
2.4 Pipeline di Popolamento	6
2.5 Statistiche Riassuntive	7
3 Classi dello Schema dell'Ontologia	7
3.1 Gerarchia delle Classi	7
3.2 Tabella Completa delle Classi Primitive	8
3.3 Scelte di Modellazione	10
3.3.1 Partizione di Platform	10
3.3.2 Generi come Classi Aperte	10
3.3.3 GameMode come Classe Top-Level Separata	10
3.3.4 Review e GameEvent: Pattern N-ary Relation e Lifecycle Event	10
4 Proprietà dello Schema dell'Ontologia	11
4.1 Object Property	11
4.2 Datatype Property	12
4.3 Property Chain Axiom	12
4.4 Proprietà Transitive	13
4.5 Coppie Inverse Sistematiche	13
4.6 Chiavi (owl:hasKey)	14
5 Classi Definite	14
5.1 Panoramica	14
5.2 Analisi per Categoria	15
5.2.1 Mode-derived	15
5.2.2 Platform-derived	15
5.2.3 Sequel-derived	16
5.2.4 Quality-derived	16
5.2.5 Award e Franchise (preesistenti)	16
5.2.6 Cardinality (solo necessarie)	16
5.3 Costrutti OWL 2 Utilizzati	16
6 Disjointness	17
6.1 Blocchi AllDisjointClasses	17
6.2 Importanza delle Disjointness	17
6.3 Nota su SinglePlayerGame e MultiplayerGame	17

7	Allineamento Esterno	18
7.1	equivalentClass verso Wikidata	18
7.2	equivalentProperty verso Dublin Core	18
7.3	Vantaggi dell'Allineamento	18
7.4	Limitazione Attuale	18
8	Profilo OWL 2 ed Espressività DL	19
8.1	Espressività: $SRIN^{(D)}$	19
8.2	Profili OWL 2: perché serve il reasoning a due livelli	19
8.3	Vincoli globali di OWL 2 DL	20
8.4	Limiti Noti	20
9	Reasoning	20
9.1	Architettura a Due Livelli	20
9.1.1	Livello 1: owlrl (OWL 2 RL)	20
9.1.2	Livello 2: construct_reasoner.py (SPARQL CONSTRUCT)	21
9.2	Perché Alcuni Costrutti Non Sono in OWL 2 RL	21
9.3	Esempio di Inferenza a Due Livelli	22
9.4	Bug Critico: owl:sameAs Falsi e Soluzione	22
9.4.1	Il Problema	22
9.4.2	La Soluzione	23
9.4.3	Impatto	23
9.5	Materialiser Mirato in enrich_owl.py	23
10	Interrogare l'Ontologia con SPARQL	24
10.1	Classi Inferite al Posto dei Filtri	24
10.2	Relazioni Derivate da Property Chain	24
10.3	Transitività: Materializzazione o Property Path	24
10.4	Navigazione Tramite Inverse	25
10.5	Open World e Negazione	25
11	Popolamento dell'Ontologia	25
11.1	Pipeline populate_wikidata.py	25
11.1.1	Query SPARQL verso Wikidata	25
11.1.2	Deduplicazione	26
11.1.3	OWL-RL Reasoning	26
11.1.4	Pruning	26
11.2	Demo Subset 2020–2026	26
11.3	Performance: rdflib vs pyoxigraph	26
12	Architettura del Sistema	27
12.1	Diagramma Architetture	27
12.2	Stack Tecnologico	27
12.3	Flusso Dati	27
12.4	Cache a Due Livelli	28
12.5	Tabella dei Componenti	28
12.6	API Endpoint	28
13	Frontend e Backend	28
13.1	Frontend: React + TypeScript + Vite	28
13.1.1	Componenti Principali	29
13.1.2	Caratteristiche del Grafo Interattivo	29
13.2	Backend: FastAPI Async	29

13.2.1	SPARQL Agent	30
13.2.2	GraphBuilder	30
13.2.3	Cache a Due Livelli	30
13.3	Endpoint API	30
14	Conclusione	30
14.1	Riepilogo dei Risultati	30
14.2	Possibili Estensioni Future	31
A	Esempio di Istanze di Test	31
A.1	Risultati della Verifica	32
B	Sintassi DL delle Classi Definite	32
C	Assiomi dello Schema in Turtle	33
	Bibliografia	35

1 Introduzione

1.1 Contesto: Semantic Web e Ontologie

Il Semantic Web, introdotto da Tim Berners-Lee nel 2001, rappresenta un'estensione del Web tradizionale in cui l'informazione è strutturata e dotata di semantica formale, consentendo alle macchine di interpretare e ragionare sui dati in modo automatico. Il fondamento teorico di questa visione risiede nel concetto di **ontologia**, definita da Gruber [1993] come <una specifica esplicita e formale di una concettualizzazione condivisa>.

Le ontologie, espresse in linguaggi formali come OWL 2 [Hitzler et al., 2012], permettono di modellare un dominio di conoscenza attraverso:

- **Classi** — categorie di entità (es. `VideoGame`, `Developer`);
- **Proprietà** — relazioni tra entità (es. `developedBy`) e attributi (es. `metacriticScore`);
- **Assiomi** — vincoli logici che arricchiscono la semantica (es. classi definite, disjointness, catene di proprietà).

Un knowledge graph è la concretizzazione pratica di un'ontologia popolata con dati reali: nodi (istanze) e archi (asserzioni RDF) formano un grafo interrogabile tramite query SPARQL.

1.2 Obiettivo del Progetto

Il progetto **Videogame Semantic Search** si propone di:

1. **Costruire un'ontologia OWL 2 di alta qualità** sul dominio dei videogiochi, coprendo il periodo 2010–2026, con uno schema ricco di assiomi (classi primitive e definite, disjointness, property chain, allineamenti esterni);
2. **Popolare l'ontologia da Wikidata** tramite query SPARQL all'endpoint pubblico di Wikidata, producendo un insieme di istanze di oltre 1.1 milioni di triple;
3. **Abilitare l'interrogazione in linguaggio naturale** attraverso un agente SPARQL basato su Large Language Model (GPT-4.1-mini) che traduce domande in italiano (o inglese) in query SPARQL eseguibili;
4. **Visualizzare i risultati come grafo di conoscenza interattivo**, navigabile con zoom, pan, filtri e dettaglio nodi.

1.3 Casi d'Uso

I principali casi d'uso del sistema sono:

- **Ricerca semantica** — interrogare il knowledge graph con domande come <Quali giochi ha sviluppato FromSoftware?> o <Top 10 giochi RPG con Metacritic più alto>;
- **Grafo di conoscenza interattivo** — esplorare visualmente le relazioni tra giochi, sviluppatori, generi, piattaforme e franchise;
- **Reasoning automatico** — classificare automaticamente i giochi in categorie derivate (MultiplayerGame, HighlyRatedGame, SequelGame, ecc.) sfruttando gli assiomi OWL 2.

1.4 Panoramica della Relazione

Il presente documento è organizzato come segue. Le Sezioni 2–8 costituiscono il cuore della relazione e analizzano in dettaglio l'ontologia: panoramica, classi, proprietà e chiavi, classi definite, disjointness, allineamento esterno, e infine espressività DL e profilo OWL 2. La Sezione 9 approfondisce il sistema di reasoning a due livelli con un esempio completo di inferenza, incluso il problema degli `owl:sameAs` falsi e la sua risoluzione; la Sezione 10 mostra come interrogare con SPARQL la conoscenza inferita. Le Sezioni 11, 12 e 13 descrivono il contesto: la pipeline di popolamento da Wikidata, l'architettura software e l'applicazione web. La Sezione 14 riassume i risultati e delinea possibili sviluppi futuri. Le appendici riportano le istanze di test, la sintassi DL delle classi definite e gli assiomi principali dello schema in Turtle.

2 Panoramica dell'Ontologia

2.1 Dominio

L'ontologia modella il dominio dei **videogiochi** nel periodo 2010–2026, coprendo le seguenti entità principali:

- **VideoGame** — il prodotto videoludico con i suoi attributi (nome, data rilascio, punteggio Metacritic, paese d'origine);
- **Developer / Publisher** — gli studi di sviluppo e le case editrici;
- **Genre** — il genere (Action, RPG, Strategy, ...);
- **Platform** — la piattaforma di esecuzione (PS5, PC, Switch, ...);
- **Character** — i personaggi (giocanti e non);
- **Franchise** — le serie e i sotto-franchise;
- **Award** — i premi ricevuti;
- **GameEngine** — il motore grafico utilizzato;
- **GameMode** — la modalità di gioco (single-player, multiplayer, co-op, MMO);
- **Review / GameEvent** — recensioni ed eventi del ciclo di vita.

2.2 Namespace e Separazione Schema/Istanze

Il namespace dell'ontologia è:

`http://www.videogame-ontology.org/ontology#`

abbreviato in **vg:** in tutte le query SPARQL e nei riferimenti interni.

L'ontologia è strutturata in due file distinti che realizzano la classica separazione schema/istanze:

- **videogames.owl** — **schema**: contiene tutte le dichiarazioni di classi, proprietà, assiomi, disjointness e allineamenti esterni. Circa 934 triple;
- **videogames_pruned_2020.owl** — **insieme di istanze di demo**: contiene le istanze (giochi, developer, publisher, generi, piattaforme, ...) per il periodo 2020–2026. Circa 745.000 triple, 68.700 giochi.

2.3 Fonte Dati: Wikidata

Tutti i dati provengono da **Wikidata** [Foundation, 2024], il knowledge graph libero della Wikimedia Foundation. La popolazione avviene tramite query SPARQL all'endpoint <https://query.wikidata.org/sparql>, mappando le proprietà Wikidata rilevanti (identificatori P31, P136, P400, P577, P444, ...) nelle corrispondenti proprietà dell'ontologia.

2.4 Pipeline di Popolamento

La pipeline completa si articola in quattro fasi:

1. **populate_wikidata.py** — esegue 204 iterazioni (17 anni × 12 mesi) di query SPARQL verso Wikidata, raccogliendo giochi, metadati e relazioni. Tempo stimato: ~5 ore con polite delay di 1 secondo tra le richieste.
2. **OWL-RL closure via owlrl** — applica la chiusura deduttiva OWL 2 RL per materializzare proprietà inverse, catene `subPropertyOf`, e inferenze da `someValuesFrom` e `intersectionOf`. Da ~1M a ~1.4M triple.
3. **Pruning** — rimuove triple ridondanti o inutilizzate a runtime: `owl:sameAs` riflessivi, `gameDescription`, `officialWebsite`. Da ~1.4M a ~1.17M triple (completo) o ~745k (demo 2020+).

4. `enrich_owl.py` — gestisce gli assiomi DL-only (FunctionalProperty sulle datatype property e `hasKey`, cfr. Sezione 9): li rimuove dalla copia di lavoro prima del reasoning e, con l'opzione `--keep-dl-axioms`, li ripristina nel file serializzato.

2.5 Statistiche Riassuntive

La Tabella 1 riassume le principali metriche dello schema dell'ontologia.

Tabella 1: Statistiche dello schema dell'ontologia

Metrica	Valore
Classi totali	78
Classi primitive	61
Classi definite (<code>equivalentClass</code>)	15
Classi con vincolo di cardinalità	2
Object Property	31
Datatype Property	26
Property chain axioms	3
Proprietà transitive	3
Proprietà simmetriche	3
Coppie inverse	12
Blocchi <code>AllDisjointClasses</code>	12
Vincoli <code>hasKey</code>	3
Allineamenti <code>equivalentClass</code>	4 (Wikidata)
Allineamenti <code>equivalentProperty</code>	3 (Dublin Core)
Triple dello schema totali	~ 934

Lo schema è espresso in formato RDF/XML; l'Appendice C ne riporta gli assiomi principali in Turtle. L'espressività DL e il profilo OWL 2 sono discussi nella Sezione 8, il reasoning nella Sezione 9.

3 Classi dello Schema dell'Ontologia

3.1 Gerarchia delle Classi

L'ontologia definisce **12 classi top-level**, ciascuna corrispondente a un'entità fondamentale del dominio, più un ricco albero di sottoclassi che raggiunge un totale di **78 classi**: 61 primitive, 15 definite tramite `owl:equivalentClass` e 2 caratterizzate da un vincolo di cardinalità massima (condizione solo necessaria).

Le 12 classi top-level sono:

1. `vg:VideoGame` — il videogioco, classe centrale dell'ontologia;
2. `vg:Developer` — lo studio di sviluppo;
3. `vg:Publisher` — l'editore;
4. `vg:Genre` — il genere videoludico;
5. `vg:Platform` — la piattaforma hardware/software;
6. `vg:Character` — il personaggio (giocante o non);
7. `vg:Franchise` — la serie/franchise;
8. `vg:Award` — il premio;
9. `vg:GameEngine` — il motore grafico;
10. `vg:GameMode` — la modalità di gioco;
11. `vg:Review` — la recensione;
12. `vg:GameEvent` — l'evento del ciclo di vita.

3.2 Tabella Completa delle Classi Primitive

La Tabella 2 elenca tutte le 61 classi primitive (escluse le 15 definite e le 2 con vincolo di cardinalità, trattate nella Sezione 5), raggruppate per classe top-level di appartenenza.

Tabella 2: Classi primitive dell'ontologia

Classe	Parent	Descrizione
VideoGame (classe top-level)		
VideoGame	owl:Thing	Prodotto videoludico
ReleasedGame	VideoGame	Gioco ufficialmente rilasciato
EarlyAccessGame	VideoGame	Gioco in accesso anticipato
CancelledGame	VideoGame	Gioco cancellato prima del rilascio
DelistedGame	VideoGame	Gioco rimosso dagli store digitali
IndieGame	VideoGame	Gioco a budget ridotto, sviluppatore indipendente
AAAGame	VideoGame	Gioco ad alto budget, grande publisher
GOTYWinner	AwardWinningGame	Vincitore del premio Game of the Year
Developer (classe top-level)		
Developer	owl:Thing	Studio di sviluppo
FirstPartyDeveloper	Developer	Studio proprietario di un platform holder
ThirdPartyDeveloper	Developer	Studio indipendente
IndieDeveloper	Developer	Piccolo sviluppatore senza publisher
Publisher (classe top-level)		
Publisher	owl:Thing	Casa editrice
FirstPartyPublisher	Publisher	Divisione editoriale di un platform holder
ThirdPartyPublisher	Publisher	Casa editrice indipendente
Genre (classe top-level)		
Genre	owl:Thing	Genere videoludico
ActionGenre	Genre	Azione, sfide fisiche
RPGGenre	Genre	Gioco di ruolo, progressione
StrategyGenre	Genre	Strategia, pensiero tattico
PuzzleGenre	Genre	Rompicapo, problem-solving
SimulationGenre	Genre	Simulazione di sistemi reali/fittizi
SportsGenre	Genre	Sport, competizioni atletiche
RacingGenre	Genre	Corse, competizione su veicoli
AdventureGenre	Genre	Esplorazione, narrativa
HorrorGenre	Genre	Horror, paura, suspense
PlatformerGenre	Genre	Piattaforme, ostacoli

Continua nella pagina successiva

Tabella 2 – continua dalla pagina precedente

Classe	Parent	Descrizione
Platform (classe top-level)		
Platform	owl:Thing	Piattaforma di esecuzione
ConsolePlatform	Platform	Console dedicata
PlayStationPlatform	ConsolePlatform	Famiglia Sony PlayStation
XboxPlatform	ConsolePlatform	Famiglia Microsoft Xbox
NintendoPlatform	ConsolePlatform	Famiglia Nintendo
PCPlatform	Platform	Personal computer
WindowsPlatform	PCPlatform	Microsoft Windows
MacOSPlatform	PCPlatform	Apple macOS
LinuxPlatform	PCPlatform	Linux
MobilePlatform	Platform	Dispositivo mobile
IOSPlatform	MobilePlatform	Apple iOS
AndroidPlatform	MobilePlatform	Android
Character (classe top-level)		
Character	owl:Thing	Personaggio
PlayerCharacter	Character	Controllato dal giocatore
NonPlayerCharacter	Character	Controllato dall'IA
Franchise (classe top-level)		
Franchise	owl:Thing	Serie/franchise
SubFranchise	Franchise	Spin-off o sotto-serie
Award (classe top-level)		
Award	owl:Thing	Premio
IndustryAward	Award	Conferito da ente industriale
EditorialAward	Award	Conferito da testata editoriale
UserAward	Award	Determinato da voto popolare
GameEngine (classe top-level)		
GameEngine	owl:Thing	Motore grafico
ProprietaryEngine	GameEngine	Proprietario, non pubblico
OpenSourceEngine	GameEngine	Open-source
GameMode (classe top-level)		
GameMode	owl:Thing	Modalità di gioco
SinglePlayerMode	GameMode	Giocatore singolo
MultiplayerMode	GameMode	Multigiocatore
CoopMode	GameMode	Cooperativa
MMOMode	GameMode	Massively Multiplayer Online
Review (classe top-level)		
Review	owl:Thing	Recensione (relazione N-aria gioco–revisore–punteggio)
GameEvent (classe top-level)		
GameEvent	owl:Thing	Evento del ciclo di vita
AnnouncementEvent	GameEvent	Annuncio pubblico
ReleaseEvent	GameEvent	Rilascio ufficiale

Continua nella pagina successiva

Tabella 2 – continua dalla pagina precedente

Classe	Parent	Descrizione
UpdateEvent	GameEvent	Aggiornamento/patch
DelistingEvent	GameEvent	Rimozione dagli store

3.3 Scelte di Modellazione

3.3.1 Partizione di Platform

La classe **Platform** è stata partizionata in tre rami principali (**ConsolePlatform**, **PCPlatform**, **MobilePlatform**), ciascuno ulteriormente specializzato:

Platform

```

+-- ConsolePlatform
|   +-- PlayStationPlatform
|   +-- XboxPlatform
|   +-- NintendoPlatform
+-- PCPlatform
|   +-- WindowsPlatform
|   +-- MacOSPlatform
|   +-- LinuxPlatform
+-- MobilePlatform
    +-- IOSPlatform
    +-- AndroidPlatform

```

Questa struttura consente interrogazioni a diversi livelli di granularità: <giochi disponibili su console> (**ConsolePlatform**), <giochi per PlayStation> (**PlayStationPlatform**), oppure <giochi per PC Windows> (**WindowsPlatform**). La partizione è resa semanticamente rigorosa da 4 blocchi **AllDisjointClasses** (cfr. Sezione 6). Ogni piattaforma Wikidata viene classificata al livello più specifico tramite le regole di reasoning (cfr. Sezione 9).

3.3.2 Generi come Classi Aperte

Genre ha 10 sottoclassi (**ActionGenre**, **RPGGenre**, ...) ma **non** viene dichiarata come esaustiva (**owl:oneOf**) né come coperta da un'unione. Questa scelta è deliberata: Wikidata può contenere generi non previsti dalla tassonomia attuale (es. <Battle Royale>, <Roguelike>). Un'ontologia chiusa impedirebbe l'aggiunta di nuovi generi senza modificare lo schema. Con la modellazione aperta, eventuali generi Wikidata non mappati restano istanze di **Genre** senza violare la semantica.

3.3.3 GameMode come Classe Top-Level Separata

GameMode è modellata come classe top-level separata (anziché come attributo di **VideoGame**) per consentire il pattern **Value Partition**: le sue 4 sottoclassi (**SinglePlayerMode**, **MultiplayerMode**, **CoopMode**, **MMOMode**) sono dichiarate mutuamente disgiunte, garantendo che un gioco con più modalità abbia asserzioni **hasGameMode** distinte e semanticamente non ambigue. Questa scelta permette inoltre di definire classi derivate (**SinglePlayerGame**, **MultiplayerGame**, **CoopGame**) tramite **owl:someValuesFrom** sulle modalità.

3.3.4 Review e GameEvent: Pattern N-ary Relation e Lifecycle Event

Review e **GameEvent** sono state introdotte come nuove classi top-level per modellare relazioni che richiedono reificazione:

- **Review** — reifica la relazione tra un gioco, un revisore e un punteggio. Senza la classe **Review**, sarebbe impossibile associare un punteggio (datatype property) a una specifica combinazione gioco-recensore. Segue il pattern **N-ary Relation** (cfr. Lez. 14);
- **GameEvent** — reifica eventi temporali come annunci, rilasci e aggiornamenti. Consente di associare date e metadati a eventi specifici del ciclo di vita di un gioco, seguendo il pattern **Lifecycle Event**.

4 Proprietà dello Schema dell'Ontologia

4.1 Object Property

La Tabella 3 elenca tutte le 31 object property definite nell'ontologia, incluse le inverse; la colonna **Assiomi** indica le caratteristiche OWL 2 aggiuntive.

Tabella 3: Object Property

Proprietà	Domain	Range	Assiomi	Label
developedBy	VideoGame	Developer	Asym, Irr, inv developerOf	developed by
developerOf	Developer	VideoGame	Asym, Irr	developer of
publishedBy	VideoGame	Publisher	Asym, Irr, inv publisherOf	published by
publisherOf	Publisher	VideoGame	Asym, Irr	publisher of
hasGenre	VideoGame	Genre	Asym, Irr, inv genreOf	has genre
genreOf	Genre	VideoGame	Asym, Irr	genre of
availableOn	VideoGame	Platform	Irr, inv platformFor	available on
platformFor	Platform	VideoGame	Asym, Irr	platform for
hasCharacter	VideoGame	Character	Asym, Irr, inv appearsIn	has character
appearsIn	Character	VideoGame	Asym, Irr	appears in
belongsTo	VideoGame	Franchise	Asym, Irr, inv includes	belongs to
includes	Franchise	VideoGame	Asym, Irr	includes
wonAward	VideoGame	Award	Irr, inv awardWonBy	won award
awardWonBy	Award	VideoGame	Irr	award won by
sequelOf	VideoGame	VideoGame	Trans, inv hasSequel	sequel of
hasSequel	VideoGame	VideoGame	—	has sequel
hasGameMode	VideoGame	GameMode	Irr, inv modeInGame	has game mode
modeInGame	GameMode	VideoGame	Irr	mode in game
madeWith	VideoGame	GameEngine	Asym, Irr, inv engineUsedIn	made with
engineUsedIn	GameEngine	VideoGame	Asym, Irr	engine used in
partOfFranchise	Franchise	Franchise	Trans	part of franchise
subsidiaryOf	—	—	Trans	subsidiary of
sharedFranchiseWith	VideoGame	VideoGame	Sym, chain (§4.3)	shared franchise with
sharesDeveloperWith	VideoGame	VideoGame	Sym, chain (§4.3)	shares developer with
sharesPublisherWith	VideoGame	VideoGame	Sym, chain (§4.3)	shares publisher with
reviewer	Review	—	—	reviewer
reviewedGame	Review	VideoGame	Funct, inv hasReview	reviewed game
hasReview	VideoGame	Review	—	has review
reviewOf	Review	VideoGame	—	review of
eventFor	GameEvent	VideoGame	inv hasEvent	event for
hasEvent	VideoGame	GameEvent	—	has event

Legenda: Asym = AsymmetricProperty, Irr = IrreflexiveProperty, Sym = SymmetricProperty, Trans = TransitiveProperty, Funct = FunctionalProperty, inv = inverseOf, chain = propertyChainAxiom.

In totale lo schema dichiara 13 proprietà asimmetriche, 18 irreflessive, 3 simmetriche, 3 transitive e 1 object property funzionale (**reviewedGame**). Le caratteristiche sono assegnate nel rispetto dei **vincoli globali di OWL 2 DL**: le proprietà *non semplici* — le transitive (**sequelOf**, **partOfFranchise**, **subsidiaryOf**), la loro inversa **hasSequel** e le super-proprietà di

catene (`sharedFranchiseWith`, `sharesDeveloperWith`, `sharesPublisherWith`) — non compaiono in assiomi di asimmetria, irreflessività, funzionalità, cardinalità o `hasKey`, che in OWL 2 DL sono ammessi solo su proprietà semplici per preservare la decidibilità del ragionamento.

4.2 Datatype Property

La Tabella 4 elenca tutte le 26 datatype property. Nove di esse (`gameName` e gli altri nomi specializzati) sono sotto-proprietà di `name` (`rdfs:subPropertyOf`): una query su `name` restituisce così il nome di qualsiasi entità.

Tabella 4: Datatype Property

Proprietà	Domain	Range	subPropertyOf	Label
<code>name</code>	—	<code>xsd:string</code>	—	<code>name</code>
<code>gameName</code>	<code>VideoGame</code>	<code>xsd:string</code>	<code>name</code>	<code>game name</code>
<code>gameDescription</code>	<code>VideoGame</code>	<code>xsd:string</code>	—	<code>game description</code>
<code>releaseDate</code>	<code>VideoGame</code>	<code>xsd:date</code>	—	<code>release date</code>
<code>metacriticScore</code>	<code>VideoGame</code>	<code>xsd:integer</code>	—	<code>Metacritic score</code>
<code>developerName</code>	<code>Developer</code>	<code>xsd:string</code>	<code>name</code>	<code>developer name</code>
<code>publisherName</code>	<code>Publisher</code>	<code>xsd:string</code>	<code>name</code>	<code>publisher name</code>
<code>genreName</code>	<code>Genre</code>	<code>xsd:string</code>	<code>name</code>	<code>genre name</code>
<code>platformName</code>	<code>Platform</code>	<code>xsd:string</code>	<code>name</code>	<code>platform name</code>
<code>characterName</code>	<code>Character</code>	<code>xsd:string</code>	<code>name</code>	<code>character name</code>
<code>franchiseName</code>	<code>Franchise</code>	<code>xsd:string</code>	<code>name</code>	<code>franchise name</code>
<code>awardName</code>	<code>Award</code>	<code>xsd:string</code>	<code>name</code>	<code>award name</code>
<code>awardYear</code>	<code>Award</code>	<code>xsd:integer</code>	—	<code>award year</code>
<code>description</code>	—	<code>xsd:string</code>	—	<code>description</code>
<code>officialWebsite</code>	<code>VideoGame</code>	<code>xsd:anyURI</code>	—	<code>official website</code>
<code>countryOfOrigin</code>	<code>VideoGame</code>	<code>xsd:string</code>	—	<code>country of origin</code>
<code>engineName</code>	<code>GameEngine</code>	<code>xsd:string</code>	<code>name</code>	<code>engine name</code>
<code>hasSteamAppId</code>	<code>VideoGame</code>	<code>xsd:string</code>	—	<code>Steam App ID</code>
<code>hasWikidataId</code>	—	<code>xsd:string</code>	—	<code>Wikidata ID</code>
<code>hasIGDBId</code>	<code>VideoGame</code>	<code>xsd:string</code>	—	<code>IGDB ID</code>
<code>reviewScore</code>	<code>Review</code>	<code>xsd:integer</code>	—	<code>review score</code>
<code>reviewDate</code>	<code>Review</code>	<code>xsd:date</code>	—	<code>review date</code>
<code>reviewerName</code>	<code>Review</code>	<code>xsd:string</code>	—	<code>reviewer name</code>
<code>eventDate</code>	<code>GameEvent</code>	<code>xsd:date</code>	—	<code>event date</code>
<code>budgetTier</code>	<code>VideoGame</code>	<code>xsd:string</code>	—	<code>budget tier</code>
<code>playerCount</code>	<code>VideoGame</code>	<code>xsd:integer</code>	—	<code>player count</code>

Tredici datatype property sono inoltre dichiarate `owl:FunctionalProperty` (i nove nomi specializzati, `releaseDate`, `metacriticScore`, `hasSteamAppId` e `reviewScore`); insieme ai tre vincoli `hasKey` formano la sezione di assiomi *DL-only* discussa nella Sezione 9. L'identità dei giochi è espressa tramite `hasKey` su `hasWikidataId` e `hasSteamAppId`: OWL 2 DL non ammette `owl:InverseFunctionalProperty` su datatype property.

4.3 Property Chain Axiom

L'ontologia definisce tre `owl:propertyChainAxiom`, che permettono di inferire nuove relazioni per **composizione** di proprietà esistenti. In notazione DL (Description Logic) una catena è un'*inclusione* di ruoli ($R_1 \circ R_2 \sqsubseteq S$): la composizione implica la super-proprietà, non viceversa.

1. `belongsTo` $\circ$ `includes` $\sqsubseteq$ `sharedFranchiseWith`
Due giochi condividono un franchise se il primo appartiene a un franchise e il franchise include il secondo.
2. `developedBy` $\circ$ `developerOf` $\sqsubseteq$ `sharesDeveloperWith`
Due giochi condividono lo sviluppatore se il primo è sviluppato da uno studio che ha sviluppato anche il secondo.
3. `publishedBy` $\circ$ `publisherOf` $\sqsubseteq$ `sharesPublisherWith`
Analogo per l'editore.

Tre osservazioni aiutano a leggere correttamente questi assiomi:

- **Tipizzazione.** La composizione è ben tipata: `belongsTo` va da `VideoGame` a `Franchise` e `includes` da `Franchise` a `VideoGame`, quindi `sharedFranchiseWith` collega due giochi (il suo dominio e range sono `VideoGame`). Lo stesso vale per le altre due catene.
- **Coppia riflessiva.** Poiché `includes` è l'inversa di `belongsTo`, per ogni gioco g che appartiene a un franchise f vale anche f `includes` g : la catena deriva quindi g `sharedFranchiseWith` g . Non è un errore di modellazione (la proprietà non è dichiarata irreflessiva, e non potrebbe esserlo perché non semplice), ma nelle query va escluso con `FILTER(?g != ?altro)` (Sezione 10).
- **Simmetria.** Le tre super-proprietà sono dichiarate `owl:SymmetricProperty`. Grazie alle inverse la simmetria sarebbe già implicata dalla catena stessa (da g_1 `belongsTo` f `includes` g_2 segue anche g_2 `belongsTo` f `includes` g_1); dichiararla esplicitamente documenta l'intento e la rende disponibile anche a strumenti che non materializzano le inverse.

Queste catene sono materializzate dal reasoner OWL 2 RL (`owlrl`, regola `prp-spo2`) e producono relazioni simmetriche tra giochi dello stesso franchise, studio o editore. Per evitare esplosioni combinatorie su franchise/studio/editore con migliaia di giochi, il materialiser mirato in `enrich_owl.py` limita il numero di coppie per gruppo tramite il flag `--max-group` (default 50).

4.4 Proprietà Transitive

Tre proprietà sono dichiarate transitive:

- `sequelOf` — se A è sequel di B e B è sequel di C , allora A è sequel di C . Permette di tracciare catene di sequel (es. `Dark Souls III` $\rightarrow$ `Dark Souls II` $\rightarrow$ `Dark Souls`);
- `partOfFranchise` — per gerarchie di franchise con sotto-franchise. Es. `<Mario Kart>` $\rightarrow$ `<Mario>`;
- `subsidiaryOf` — per le gerarchie societarie (studio $\rightarrow$ gruppo $\rightarrow$ holding).

La transitività di `sequelOf` è particolarmente utile per query del tipo `<tutti i sequel di Dark Souls>` senza dover navigare manualmente la catena. Poiché in OWL 2 DL una proprietà transitiva non è semplice, queste tre proprietà (e l'inversa `hasSequel`) non sono dichiarate anche asimmetriche o irreflessive: l'aciclicità delle catene di sequel e delle gerarchie di franchise resta un vincolo di qualità dei dati, non un assioma dello schema.

4.5 Coppie Inverse Sistematiche

L'ontologia definisce 12 coppie inverse, consentendo la navigazione bidirezionale del grafo. Per ogni proprietà "forward" esiste la corrispondente inversa:

- `developedBy` $\leftrightarrow$ `developerOf`
- `publishedBy` $\leftrightarrow$ `publisherOf`
- `hasGenre` $\leftrightarrow$ `genreOf`
- `availableOn` $\leftrightarrow$ `platformFor`
- `hasCharacter` $\leftrightarrow$ `appearsIn`
- `belongsTo` $\leftrightarrow$ `includes`

- `wonAward` $\leftrightarrow$ `awardWonBy`
- `sequelOf` $\leftrightarrow$ `hasSequel`
- `hasGameMode` $\leftrightarrow$ `modeInGame`
- `madeWith` $\leftrightarrow$ `engineUsedIn`
- `reviewedGame` $\leftrightarrow$ `hasReview`
- `eventFor` $\leftrightarrow$ `hasEvent`

La dichiarazione esplicita di `owl:inverseOf` in una sola direzione è sufficiente: la semantica di `owl:inverseOf` è simmetrica e `owl:rl` deriva le triple in entrambe le direzioni (regole `prp-inv1` e `prp-inv2`). In pratica basta asserire `developedBy` nei dati perché `developerOf` sia disponibile nelle query.

4.6 Chiavi (`owl:hasKey`)

Lo schema dichiara tre chiavi:

- `VideoGame owl:hasKey (hasWikidataId);`
- `VideoGame owl:hasKey (hasSteamAppId);`
- `Review owl:hasKey (reviewedGame, reviewerName).`

Semantica. Se due individui *nominati* della classe hanno gli stessi valori per tutte le proprietà della chiave, sono lo stesso individuo (`owl:sameAs`). La chiave composta di `Review` esprime che un revisore scrive al più una recensione per gioco: due recensioni dello stesso revisore sullo stesso gioco vengono identificate.

Perché `hasKey` e non `InverseFunctionalProperty`. Una proprietà inversamente funzionale avrebbe lo stesso effetto (valore condiviso $\Rightarrow$ stesso individuo), ma su una datatype property richiederebbe l'inversa di una data property, che in OWL 2 DL non esiste. `hasKey` è invece *DL-safe*: vale solo per gli individui esplicitamente nominati e non per quelli anonimi introdotti dal ragionamento, per cui può essere aggiunta senza compromettere la decidibilità (Sezione 8).

Le tre chiavi, insieme alle datatype property funzionali, fanno parte della sezione di assiomi *DL-only*, che viene rimossa prima del reasoning OWL 2 RL per evitare `owl:sameAs` spuri (Sezione 9).

5 Classi Definite

5.1 Panoramica

Le classi definite (*defined classes*) costituiscono uno degli aspetti più avanzati dell'ontologia. A differenza delle classi primitive, che richiedono l'asserzione esplicita `rdf:type`, le classi definite specificano condizioni necessarie e sufficienti (`owl:equivalentClass`) che permettono a un reasoner di **inferire automaticamente** l'appartenenza di un'istanza.

L'ontologia definisce **15 classi definite**, raggruppabili in cinque categorie logiche, a cui si affiancano **2 classi con vincolo di cardinalità** dotate di sole condizioni necessarie. La Tabella 5 le elenca tutte e 17 con la sintassi DL e il tipo di condizione.

Tabella 5: Classi definite e classi con vincolo di cardinalità

Classe	Tipo	Sintassi DL	Costrutti OWL 2
MultiplayerGame	S/N	<code>VideoGame $\sqcap$ $\exists$hasGameMode.MultiplayerMode</code>	<code>intersectionOf, someValuesFrom</code>
SinglePlayerGame	S/N	<code>VideoGame $\sqcap$ $\exists$hasGameMode.SinglePlayerMode</code>	<code>intersectionOf, someValuesFrom</code>
CoopGame	S/N	<code>VideoGame $\sqcap$ $\exists$hasGameMode.CoopMode</code>	<code>intersectionOf, someValuesFrom</code>
PCGame	S/N	<code>VideoGame $\sqcap$ $\exists$availableOn.PCPlatform</code>	<code>intersectionOf, someValuesFrom</code>

Tabella 5 – continua

Classe	Tipo	Sintassi DL	Costrutti OWL 2
ConsoleGame	S/N	$\text{VideoGame} \sqcap \exists \text{availableOn}.\text{ConsolePlatform}$	intersectionOf, someValuesFrom
MobileGame	S/N	$\text{VideoGame} \sqcap \exists \text{availableOn}.\text{MobilePlatform}$	intersectionOf, someValuesFrom
NintendoGame	S/N	$\text{VideoGame} \sqcap \exists \text{availableOn}.\text{NintendoPlatform}$	intersectionOf, someValuesFrom
PlayStationGame	S/N	$\text{VideoGame} \sqcap \exists \text{availableOn}.\text{PlayStationPlatform}$	intersectionOf, someValuesFrom
XboxGame	S/N	$\text{VideoGame} \sqcap \exists \text{availableOn}.\text{XboxPlatform}$	intersectionOf, someValuesFrom
SequelGame	S/N	$\text{VideoGame} \sqcap \exists \text{sequelOf}.\text{VideoGame}$	intersectionOf, someValuesFrom
StandaloneGame	S/N	$\text{VideoGame} \sqcap \neg \exists \text{sequelOf}.\text{VideoGame}$	intersectionOf, complementOf
HighlyRatedGame	S/N	$\text{VideoGame} \sqcap \exists \text{metacriticScore}.\text{integer}[\geq 85]$	intersectionOf, someValuesFrom, datatype restriction
RecentGame	S/N	$\text{VideoGame} \sqcap \exists \text{releaseDate}.\text{date}[\geq 2020-01-01]$	intersectionOf, someValuesFrom, datatype restriction
AwardWinningGame	S/N	$\text{VideoGame} \sqcap \exists \text{wonAward}.\text{Award}$	intersectionOf, someValuesFrom
FranchiseGame	S/N	$\text{VideoGame} \sqcap \exists \text{belongsTo}.\text{Franchise}$	intersectionOf, someValuesFrom
GameWithSingleDeveloper	N	$\sqsubseteq \text{VideoGame} \sqcap \leq 1 \text{developedBy}$	subClassOf, maxCardinality
ExclusiveRelease	N	$\sqsubseteq \text{VideoGame} \sqcap \leq 1 \text{availableOn}$	subClassOf, maxCardinality

Legenda: S/N = condizioni sufficienti e necessarie (equivalentClass); N = solo necessarie (subClassOf).

5.2 Analisi per Categoria

5.2.1 Mode-derived

Le classi `MultiplayerGame`, `SinglePlayerGame` e `CoopGame` classificano i giochi in base alla modalità di gioco. Un gioco viene inferito come `MultiplayerGame` se esiste almeno un'asserzione `hasGameMode` verso un'entità di tipo `MultiplayerMode`. Si noti che `SinglePlayerGame` e `MultiplayerGame` **non** sono dichiarate disgiunte: un gioco può supportare entrambe le modalità (es. *Call of Duty* ha sia campagna single-player che multiplayer).

Esempio: *Elden Ring* ha `hasGameMode` verso un individuo di tipo `SinglePlayerMode` e uno di tipo `MultiplayerMode` → inferito sia come `SinglePlayerGame` che come `MultiplayerGame`.

5.2.2 Platform-derived

Sei classi classificano i giochi per piattaforma a due livelli di granularità: macrocategoria (`PCGame`, `ConsoleGame`, `MobileGame`) e famiglia specifica (`NintendoGame`, `PlayStationGame`, `XboxGame`). L'inferenza sfrutta la gerarchia `rdfs:subClassOf` delle piattaforme: se una piattaforma è di tipo `PlayStationPlatform`, il reasoner inferisce anche `ConsolePlatform` e `Platform`.

Esempio: *God of War Ragnarök* ha `availableOn PS5` (tipo `PlayStationPlatform`) → inferito `PlayStationGame` e `ConsoleGame`.

5.2.3 Sequel-derived

`SequelGame` e `StandaloneGame` formano una partizione completa dei giochi basata sulla proprietà transitiva `sequelOf`:

- `SequelGame` — $\exists \text{sequelOf.VideoGame}$: il gioco ha almeno un predecessore diretto;
- `StandaloneGame` — $\neg \exists \text{sequelOf.VideoGame}$: il gioco non ha predecessori. Utilizza il costrutto `owl:complementOf`, non supportato da OWL 2 RL, e richiede pertanto il reasoner CONSTRUCT (cfr. Sezione 9).

Esempio: Elden Ring non ha `sequelOf` → `StandaloneGame`; Dark Souls III ha `sequelOf` Dark Souls II → `SequelGame`.

5.2.4 Quality-derived

`HighlyRatedGame` e `RecentGame` utilizzano **datatype restriction** con `xsd:minInclusive`, un costrutto OWL 2 DL non coperto da OWL 2 RL (cfr. Lez. 17–18):

- `HighlyRatedGame` $\equiv$ `VideoGame` $\sqcap$ $\exists \text{metacriticScore.xsd:integer}[\geq 85]$ — giochi con Metacritic ≥ 85 ;
- `RecentGame` $\equiv$ `VideoGame` $\sqcap$ $\exists \text{releaseDate.xsd:date}[\geq 2020-01-01]$ — giochi rilasciati dal 2020.

Esempio: Elden Ring (metacritic 96) → `HighlyRatedGame`.

5.2.5 Award e Franchise (preesistenti)

`AwardWinningGame` e `FranchiseGame` sono le due classi definite originali (V1), preservate nell'estensione V2:

- `AwardWinningGame`: giochi con almeno un premio (`wonAward`);
- `FranchiseGame`: giochi appartenenti a una serie (`belongsTo`).

5.2.6 Cardinality (solo necessarie)

`GameWithSingleDeveloper` e `ExclusiveRelease` sono caratterizzate da **sole condizioni necessarie** (`rdfs:subClassOf` verso una restrizione `owl:maxCardinality` non qualificata, anziché `owl:equivalentClass`): non sono quindi classi definite in senso stretto. Questa scelta è deliberata:

- `GameWithSingleDeveloper`: ≤ 1 `developedBy`. Una condizione sufficiente sarebbe di scarsa utilità: per la *Open World Assumption* un reasoner può concludere ≤ 1 `developedBy` solo in presenza di assiomi di chiusura espliciti, per cui la classe non classificherebbe quasi mai alcun gioco;
- `ExclusiveRelease`: ≤ 1 `availableOn`. Analogo ragionamento: un gioco esclusiva *ha al massimo* 1 piattaforma, ma non tutti i giochi con ≤ 1 piattaforma sono necessariamente esclusive.

Queste classi forniscono vincoli di integrità piuttosto che meccanismi di classificazione automatica.

5.3 Costrutti OWL 2 Utilizzati

La Tabella 6 riassume i costrutti OWL 2 impiegati nelle classi definite, con riferimento alle lezioni del corso. La serializzazione RDF di questi costrutti (nodi blank per restrizioni, liste per intersezioni) è mostrata nell'Appendice C.

Tabella 6: Costrutti OWL 2 nelle classi definite

Costrutto	Classi	Rif. Lezione
owl:someValuesFrom	14: 12 su object property, 2 su datatype property	Lez. 15 (esistenziale)
owl:intersectionOf	Tutte le 15 classi definite	Lez. 16 (intersezione)
owl:complementOf	StandaloneGame	Lez. 16 (complemento)
owl:maxCardinality	GameWithSingleDeveloper, ExclusiveRelease	Lez. 17–18
owl:withRestrictions	HighlyRatedGame, RecentGame	Lez. 17–18 (datatype)
xsd:minInclusive	HighlyRatedGame, RecentGame	Lez. 17–18

6 Disjointness

6.1 Blocchi AllDisjointClasses

L'ontologia dichiara **12 blocchi** owl:AllDisjointClasses, elencati nella Tabella 7. Ciascun blocco specifica un insieme di classi mutuamente disgiunte: nessuna istanza può appartenere a più di una classe nello stesso blocco.

Tabella 7: Blocchi AllDisjointClasses

#	Classi
1	VideoGame, Developer, Publisher, Genre, Platform, Character, Franchise, Award, GameEngine, GameMode, Review, GameEvent
2	ConsolePlatform, PCPlatform, MobilePlatform
3	PlayStationPlatform, XboxPlatform, NintendoPlatform
4	WindowsPlatform, MacOSPlatform, LinuxPlatform
5	IOSPlatform, AndroidPlatform
6	IndustryAward, EditorialAward, UserAward
7	PlayerCharacter, NonPlayerCharacter
8	ProprietaryEngine, OpenSourceEngine
9	ReleasedGame, EarlyAccessGame, CancelledGame, DelistedGame
10	IndieGame, AAAGame
11	SinglePlayerMode, MultiplayerMode, CoopMode, MMOMode
12	AnnouncementEvent, ReleaseEvent, UpdateEvent, DelistingEvent

6.2 Importanza delle Disjointness

Le dichiarazioni di disjointness svolgono due ruoli fondamentali:

1. **Rilevazione di inconsistenze nei dati Wikidata.** Se un'entità Wikidata venisse erroneamente classificata come, ad esempio, sia **ConsolePlatform** che **PCPlatform**, un reasoner DL completo (HermiT, Pellet) rileverebbe l'inconsistenza. Questo meccanismo funge da *sanity check* automatico sulla qualità dei dati;
2. **Ragionamento DL.** Le disjointness abilitano inferenze per contraddizione. Ad esempio, sapendo che **PlayerCharacter** e **NonPlayerCharacter** sono disgiunte, se un personaggio viene classificato come **PlayerCharacter**, si può inferire che **non** è **NonPlayerCharacter**.

6.3 Nota su SinglePlayerGame e MultiplayerGame

Si noti che **SinglePlayerGame** e **MultiplayerGame** **non** compaiono in alcun blocco di disjointness. La scelta è deliberata: nel dominio dei videogiochi, è comune che un gioco supporti sia la modalità single-player che multiplayer (es. Grand Theft Auto V, Elden Ring, Call of Duty). Dichiararle disgiunte produrrebbe inconsistenze su una porzione significativa delle istanze.

Al contrario, le modalità `SinglePlayerMode`, `MultiplayerMode`, `CoopMode` e `MMOMode` (blocco #11) sono disgiunte: una specifica modalità di gioco è esattamente una di queste quattro.

7 Allineamento Esterno

7.1 `equivalentClass` verso Wikidata

L'ontologia dichiara 4 allineamenti `owl:equivalentClass` verso entità Wikidata, riportati nella Tabella 8.

Tabella 8: Allineamenti `equivalentClass` verso Wikidata

Classe VG	Entità Wikidata	Descrizione Wikidata
VideoGame	wd:Q7889	Video game
Developer	wd:Q210167	Video game developer
GameEngine	wd:Q193564	Game engine
Genre	wd:Q65956376	Video game genre

7.2 `equivalentProperty` verso Dublin Core

Tre datatype property sono allineate con il vocabolario Dublin Core [Initiative, 2020] tramite `owl:equivalentProperty` (Tabella 9).

Tabella 9: Allineamenti `equivalentProperty` verso Dublin Core

Proprietà VG	Proprietà DC	Label DC
releaseDate	dcterms:issued	Date Issued
gameName	dcterms:title	Title
gameDescription	dcterms:description	Description

7.3 Vantaggi dell'Allineamento

L'allineamento esterno, noto in letteratura come **ontology alignment** (cfr. Lez. 13), offre tre vantaggi principali:

1. **Interoperabilità.** Un'applicazione che consuma dati Dublin Core può automaticamente interpretare `vg:releaseDate` come sinonimo di `dcterms:issued`, senza bisogno di mapping manuali;
2. **Linkage automatico.** Le entità Wikidata esterne referenziate tra le istanze (es. `wd:Q12345`) sono automaticamente riconosciute come istanze di `vg:VideoGame` se Wikidata le classifica come `Q7889`;
3. **Riuso di standard consolidati.** Aniché reinventare vocabolari, l'ontologia si appoggia a standard ampiamente adottati (Dublin Core per metadati bibliografici, Wikidata per identificazione entità), seguendo il principio *Linked Data* di riutilizzare URI esistenti.

7.4 Limitazione Attuale

L'allineamento è dichiarato a livello dello schema ma non viene sfruttato dal reasoner OWL 2 RL, poiché `owl:rl` opera solo sul namespace locale. Per sfruttare appieno questi assiomi, sarebbe necessario un reasoner DL completo (HermiT, Pellet) che possa importare anche le definizioni di classe da Wikidata.

8 Profilo OWL 2 ed Espressività DL

Questa sezione colloca l'ontologia rispetto alla teoria delle Description Logic e ai profili di OWL 2, e spiega perché lo schema è progettato per OWL 2 DL mentre il reasoning pratico usa il profilo OWL 2 RL (Sezione 9).

8.1 Espressività: $\mathcal{SRIN}^{(\mathcal{D})}$

OWL 2 DL corrisponde alla logica descrittiva $\mathcal{SROIQ}^{(\mathcal{D})}$. I costrutti effettivamente usati nello schema collocano l'ontologia nel frammento $\mathcal{SRIN}^{(\mathcal{D})}$. La Tabella 10 associa a ogni lettera il costrutto e il punto dello schema che lo richiede.

Tabella 10: Espressività DL dell'ontologia

Lettera	Costrutto	Uso nello schema
$\mathcal{ALC}$	$\sqcap, \sqcup, \neg$ (e $\forall, \sqsubseteq$ per dualità)	classi definite <code>VideoGame</code> $\sqcap \exists p.C$; complemento in <code>StandaloneGame</code>
$\mathcal{S}$	$\mathcal{ALC}$ + ruoli transitivi	<code>sequelOf</code> , <code>partOfFranchise</code> , <code>subsidiaryOf</code>
$\mathcal{R}$	inclusioni complesse di ruoli, asimmetria, irreflessività	3 property chain; 13 proprietà asimmetriche e 18 irreflessive
$\mathcal{I}$	ruoli inversi	12 <code>owl:inverseOf</code> ; 3 proprietà simmetriche ($R \equiv R^-$)
$\mathcal{N}$	restrizioni di cardinalità non qualificate	<code>≤1 developedBy</code> , <code>≤1 availableOn</code> ; <code>reviewedGame</code> funzionale
$(\mathcal{D})$	datatype, facet, data property funzionali	26 datatype property; <code>minInclusive</code> in <code>HighlyRatedGame</code> e <code>RecentGame</code>

Rispetto a $\mathcal{SROIQ}^{(\mathcal{D})}$ mancano due componenti:

- $\mathcal{O}$ (nominali): lo schema non usa `owl:oneOf` né `owl:hasValue`; le categorie (generi, piattaforme, modalità) sono modellate come classi e non come insiemi chiusi di individui;
- $\mathcal{Q}$ (cardinalità qualificate): le due restrizioni di cardinalità non specificano `owl:onClass`.

Gli assiomi `owl:hasKey` non modificano l'espressività: sono *DL-safe*, cioè si applicano solo agli individui nominati (Sezione 4.6).

8.2 Profili OWL 2: perché serve il reasoning a due livelli

OWL 2 definisce tre profili trattabili (EL, QL, RL). L'ontologia non appartiene a nessuno di essi:

- **OWL 2 EL** esclude ruoli inversi, complemento e cardinalità;
- **OWL 2 QL** esclude, tra l'altro, le property chain e la transitività;
- **OWL 2 RL** ammette $\exists p.C$ solo come *sottoclasse* e non ammette datatype restriction né complemento in posizione di sottoclasse.

Il caso di OWL 2 RL è quello rilevante per il progetto. Un'equivalenza $X \equiv \text{VideoGame} \sqcap \exists p.C$ contiene due inclusioni:

- $\text{VideoGame} \sqcap \exists p.C \sqsubseteq X$ (condizione sufficiente): ammessa in RL, è quella che serve per *classificare* (regole `cls-int1` e `cls-svf1`);
- $X \sqsubseteq \exists p.C$ (condizione necessaria): non ammessa in RL, perché richiederebbe di introdurre individui anonimi.

Un reasoner RL come `owlrl` applica quindi correttamente la direzione utile alla classificazione e ignora l'altra. Restano invece fuori dal profilo, in entrambe le direzioni, le datatype restriction (`HighlyRatedGame`, `RecentGame`) e il complemento usato come condizione sufficiente (`StandaloneGame`): per questi costrutti il progetto usa il secondo livello di reasoning basato su SPARQL CONSTRUCT.

Le cardinalità massime ≤ 1 in posizione di superclasse, al contrario, sono ammesse in RL: la regola `cls-maxc2` identifica (`owl:sameAs`) i due valori di un individuo che ne avesse più di uno.

Non producono però classificazioni, perché `GameWithSingleDeveloper` ed `ExclusiveRelease` hanno sole condizioni necessarie.

8.3 Vincoli globali di OWL 2 DL

Per garantire la decidibilità, OWL 2 DL impone vincoli globali sull'uso delle proprietà. Una proprietà è *non semplice* se è transitiva, se è super-proprietà di una catena, oppure se è inversa o super-proprietà di una proprietà non semplice. Nello schema le proprietà non semplici sono sette: `sequalOf`, `hasSequel`, `partOfFranchise`, `subsidiaryOf`, `sharedFranchiseWith`, `sharesDeveloperWith`, `sharesPublisherWith`.

I vincoli rispettati dallo schema sono:

1. **Proprietà semplici:** nessuna proprietà non semplice compare in assiomi di asimmetria, irreflessività, funzionalità (diretta o inversa), cardinalità o `hasKey`. Per questo le tre transitive e `hasSequel` non sono dichiarate asimmetriche né irreflessive (Sezione 4);
2. **Regolarità delle catene:** le tre catene hanno la forma $R_1 \circ R_2 \sqsubseteq S$ con S distinta da R_1 e R_2 e senza dipendenze cicliche tra catene, quindi la gerarchia di ruoli è regolare;
3. **Nessuna InverseFunctionalProperty su datatype property:** OWL 2 DL non prevede l'inversa di una data property; l'identità è espressa con `hasKey`.

La verifica è stata eseguita con uno script basato su `rdflib`, che calcola l'insieme delle proprietà non semplici e ne controlla l'uso negli assiomi elencati: il risultato è di 0 violazioni.

8.4 Limiti Noti

Per l'uso con un reasoner DL completo (HermiT, Pellet) restano due aspetti:

- **Datatype `xsd:date`.** È il range di `releaseDate`, `reviewDate` ed `eventDate` e la base della restrizione di `RecentGame`, ma non fa parte della *datatype map* di OWL 2, che per gli istanti temporali prevede solo `xsd:dateTime` e `xsd:dateTimeStamp`. HermiT lo rifiuta come datatype non supportato; la soluzione è migrare schema e dati a `xsd:dateTime`;
- **Entità esterne non dichiarate.** Le entità usate negli allineamenti (`wd:*`, `dcterms:*`) non hanno una dichiarazione di tipo esplicita; strumenti come l'OWL API le dichiarano implicitamente in fase di caricamento.

9 Reasoning

9.1 Architettura a Due Livelli

Il sistema di reasoning adotta un'architettura a due livelli che combina un reasoner OWL 2 RL standard con un reasoner CONSTRUCT personalizzato per i costrutti non coperti dal profilo RL.

9.1.1 Livello 1: owlrl (OWL 2 RL)

`owlrl` [Chuc and contributors, 2024] implementa la chiusura deduttiva per il profilo OWL 2 RL [Motik et al., 2012], materializzando le triple inferite direttamente nel grafo RDF. I costrutti supportati includono:

- `rdfs:subClassOf` — propagazione del tipo verso l'alto;
- `owl:someValuesFrom` — inferenza da restrizioni esistenziali (es. se `G wonAward A`, allora `G rdfs:type AwardWinningGame`);
- `owl:hasValue` — inferenza da valore specifico;
- `owl:intersectionOf` — scomposizione di intersezioni;
- `owl:inverseOf` — materializzazione di proprietà inverse;
- `owl:SymmetricProperty` e `owl:TransitiveProperty`;

- `owl:propertyChainAxiom` — catene di proprietà.

Sull'insieme di istanze di test (3 giochi + 2 developer + piattaforme), `owlrl` produce un incremento da ~997 a ~2680 triple.

9.1.2 Livello 2: `construct_reasoner.py` (SPARQL CONSTRUCT)

21 regole CONSTRUCT colmano le lacune di OWL 2 RL, operando su costrutti che richiedono ragionamento al di fuori del profilo RL. La Tabella 11 elenca le regole.

Tabella 11: Regole CONSTRUCT del reasoner personalizzato

#	Descrizione	Cosa inferisce
R1	HighlyRatedGame da metacritic-Score ≥ 85	?g a <code>vg:HighlyRatedGame</code>
R2	RecentGame da <code>releaseDate</code> $\geq 2020-01-01$	?g a <code>vg:RecentGame</code>
R3	StandaloneGame: VideoGame senza <code>sequelOf</code>	?g a <code>vg:StandaloneGame</code> (negation-as-failure)
R4	SequelGame: VideoGame con <code>sequelOf</code>	?g a <code>vg:SequelGame</code>
R5a–f	Classificazione per piattaforma (PC, Console, Mobile, Nintendo, PS, Xbox)	?g a <code>vg:PCGame</code> , ecc.
R6a–c	Classificazione per modalità (Multiplayer, SinglePlayer, Coop)	?g a <code>vg:MultiplayerGame</code> , ecc.
R7	Inverse <code>hasSequel</code> : se A <code>sequelOf</code> B $\rightarrow$ B <code>hasSequel</code> A	?b <code>vg:hasSequel</code> ?a
R8	Inverse <code>awardWonBy</code>	?a <code>vg:awardWonBy</code> ?g
R9a–d	Inverse: <code>genreOf</code> , <code>platformFor</code> , <code>engineUsedIn</code> , <code>modeInGame</code>	completamento bidirezionale
R10	<code>sharedGenreWith</code> (pairwise)	?g1 <code>vg:sharedGenreWith</code> ?g2
R11	<code>sharedEngineWith</code> (pairwise)	?g1 <code>vg:sharedEngineWith</code> ?g2

Solo R1–R3 coprono costrutti realmente esterni a OWL 2 RL. Le regole R4–R9 (classificazione per sequel, piattaforma e modalità, completamento delle inverse) ripetono inferenze che il livello 1 già produce: sono mantenute perché il livello 2 possa essere eseguito anche da solo, senza dipendere dalla chiusura `owlrl`. R10 e R11 aggiungono relazioni di comodo tra giochi con lo stesso genere o motore.

9.2 Perché Alcuni Costrutti Non Sono in OWL 2 RL

Il profilo OWL 2 RL è deliberatamente limitato per garantire la trattabilità computazionale (polinomiale); la Sezione 8 ne discute i limiti in dettaglio. Per le classi dell'ontologia la situazione è la seguente: due famiglie di costrutti esulano dal profilo, mentre la terza vi rientra ma non produce classificazioni.

1. **Datatype restriction** (`HighlyRatedGame`, `RecentGame`). Le restrizioni su tipi di dato con `xsd:minInclusive` richiedono il confronto di valori literal, operazione non prevista dalle regole RL standard. Le regole R1 e R2 risolvono con un semplice filtro SPARQL (`FILTER(?s >= 85)`);
2. **complementOf puro** (`StandaloneGame`). OWL 2 RL supporta solo forme limitate di negazione. La regola R3 implementa la *negation-as-failure* con `FILTER NOT EXISTS`, assumendo la completezza dei dati (closed-world assumption sulla proprietà `sequelOf`);

3. **maxCardinality** (`GameWithSingleDeveloper`, `ExclusiveRelease`). Una restrizione ≤ 1 in posizione di superclasse è ammessa in OWL 2 RL (regola `cls-maxc2`: se un'istanza ha due valori, questi vengono identificati con `owl:sameAs`). Poiché le due classi hanno sole condizioni necessarie (`rdfs:subClassOf`), però, nessun gioco viene classificato in esse: fungono da vincoli di integrità, e nessuna regola CONSTRUCT è necessaria.

9.3 Esempio di Inferenza a Due Livelli

L'insieme di istanze di test (Appendice A) permette di seguire il ragionamento su un caso concreto. Il gioco `GameA` (Elden Ring) è asserito come `VideoGame` con data di rilascio 2022-02-25, punteggio Metacritic 96, sviluppatore `Dev1`, due modalità (individui di tipo `SinglePlayerMode` e `MultiplayerMode`), due piattaforme (`PS5`, di tipo `PlayStationPlatform`, e `PC_Windows`, di tipo `WindowsPlatform`) e un premio. Anche `GameC` è sviluppato da `Dev1`. La Tabella 12 riporta le conclusioni, il motivo e il livello che le produce.

Tabella 12: Inferenze su `GameA`

Conclusione	Motivo	Livello
<code>PlayStationGame</code> , <code>ConsoleGame</code>	<code>availableOn PS5</code> ; <code>PS5</code> è <code>PlayStationPlatform</code> $\sqsubseteq$ <code>ConsolePlatform</code> ; definizioni con $\exists$ <code>availableOn</code>	1
<code>PCGame</code>	<code>WindowsPlatform</code> $\sqsubseteq$ <code>PCPlatform</code>	1
<code>SinglePlayerGame</code> , <code>MultiplayerGame</code>	$\exists$ <code>hasGameMode</code> verso le due modalità	1
<code>AwardWinningGame</code>	$\exists$ <code>wonAward.Award</code> ; il premio è <code>IndustryAward</code> $\sqsubseteq$ <code>Award</code>	1
<code>HighlyRatedGame</code>	$96 \geq 85$ (datatype restriction)	2
<code>RecentGame</code>	$2022-02-25 \geq 2020-01-01$ (datatype restriction)	2
<code>StandaloneGame</code>	nessun <code>sequelOf</code> noto (negation-as-failure)	2
<code>Dev1 developerOf GameA</code>	inversa di <code>developedBy</code>	1
<code>GameA sharesDeveloperWith GameC</code>	catena <code>developedBy</code> o <code>developerOf</code> e simmetria	1

L'esempio mostra la divisione dei compiti: il livello 1 (`owlrl`) copre gerarchie, restrizioni esistenziali, inverse e catene; il livello 2 (CONSTRUCT) interviene solo dove servono confronti tra literal o negazione.

9.4 Bug Critico: `owl:sameAs` Falsi e Soluzione

9.4.1 Il Problema

Durante lo sviluppo è emerso un bug critico: `owlrl` inferiva `owl:sameAs` falsi tra giochi distinti a causa degli assiomi di identità (`FunctionalProperty`, `InverseFunctionalProperty`, `hasKey`) dichiarati su `DatatypeProperty`.

Meccanismo del bug: nel profilo OWL 2 RL le uguaglianze tra individui sono prodotte da tre regole:

- **prp-fp** (`FunctionalProperty`) — se lo *stesso soggetto* ha due valori per la proprietà, i due *valori* vengono identificati;
- **prp-ifp** (`InverseFunctionalProperty`) — se *due soggetti* condividono un valore, i due *soggetti* vengono identificati;
- **prp-key** (`hasKey`) — due individui della classe con gli stessi valori di chiave vengono identificati.

`owlrl` tratta i literal come nodi del grafo alla stregua di individui. Applicate a datatype property, **prp-ifp** e **prp-key** identificano giochi distinti che condividono il valore di un identificativo, mentre **prp-fp** produce uguaglianze tra literal diversi dello stesso gioco (es. due date di rilascio

regionali). Ogni `owl:sameAs` spurio propaga poi tipi e proprietà tra gli individui unificati (regole `eq-rep-*`), falsando la classificazione.

Esempio: nel test delle istanze (Appendice A) due `owl:sameAs` spuri collegavano `GameA` agli altri due giochi, propagando loro l’asserzione `wonAward`: prima del fix tutti e tre i giochi risultavano `AwardWinningGame`; dopo il fix solo `GameA`, l’unico con `wonAward` asserito, viene classificato come tale.

9.4.2 La Soluzione

La soluzione adottata (implementata in `enrich_owl.py`) si articola in quattro passi:

1. **Nessuna `InverseFunctionalProperty` su `datatype property`.** Oltre a innescare `prp-ifp`, questo costrutto non è ammesso in OWL 2 DL; l’identità dei giochi è espressa esclusivamente tramite `owl:hasKey` (su `hasWikidataId` e `hasSteamAppId`);
2. **Sezione “DL-only axioms” separata.** I 16 assiomi rimanenti (13 `FunctionalProperty` su `datatype property` e 3 `owl:hasKey`) sono dichiarati in una sezione dedicata in coda a `videogames.owl`, con un commento esplicativo;
3. **Strip automatico prima del reasoning.** La funzione `strip_dl_only_axioms()` rimuove questi assiomi dalla copia di lavoro in memoria prima del reasoning, prevenendo le inferenze errate;
4. **Restore opzionale.** Con il flag `--keep-dl-axioms`, la funzione `restore_dl_only_axioms()` ripristina gli assiomi nel file serializzato; per default l’output resta privo di assiomi DL-only e quindi sicuro per `owlrl`.

Lo schema `videogames.owl` conserva sempre la sezione DL-only, preservandone il valore dichiarativo per l’uso con reasoner DL completi (Hermit, Pellet).

9.4.3 Impatto

Dopo il fix: 0 `owl:sameAs` falsi su tutto l’insieme di istanze di test (2715 triple finali, nessun gioco unificato erroneamente). Le 12 famiglie di regole Python dirette in `enrich_owl.py` (`_materialise`) classificano correttamente tutti i giochi senza produrre falsi positivi.

9.5 Materialiser Mirato in `enrich_owl.py`

Oltre al reasoning `owlrl` + `CONSTRUCT`, `enrich_owl.py` implementa un materialiser Python diretto con 12 famiglie di regole che operano senza passare per SPARQL, risultando più veloce di `owlrl` full su insiemi di istanze di grandi dimensioni:

1. `AwardWinningGame` $\leftarrow$ game `wonAward` `_`;
2. `FranchiseGame` $\leftarrow$ game `belongsTo` `_`;
3. `sharedFranchiseWith` $\leftarrow$ `belongsTo` $\circ$ `includes`
4. `sharesDeveloperWith` $\leftarrow$ `developedBy` $\circ$ `developerOf`
5. `sharesPublisherWith` $\leftarrow$ `publishedBy` $\circ$ `publisherOf`
6. `MultiplayerGame` / `SinglePlayerGame` / `CoopGame` $\leftarrow$ `hasGameMode`
7. `PCGame` / `ConsoleGame` / `MobileGame` $\leftarrow$ `availableOn`
8. `NintendoGame` / `PlayStationGame` / `XboxGame` $\leftarrow$ `availableOn`
9. `HighlyRatedGame` $\leftarrow$ `metacriticScore` ≥ 85
10. `RecentGame` $\leftarrow$ `releaseDate` $\geq 2020-01-01$
11. `SequelGame` $\leftarrow$ game `sequelOf` `_`
12. `StandaloneGame` $\leftarrow$ `VideoGame` senza `sequelOf`

Le regole pairwise (3–5) sono protette dal cap `--max-group` (default 50) per evitare esplosioni combinatorie su franchise con migliaia di giochi.

10 Interrogare l'Ontologia con SPARQL

Il valore degli assiomi si vede al momento dell'interrogazione: dopo il reasoning, le conoscenze implicite sono triple esplicite del grafo e le query SPARQL possono usarle direttamente, senza ricostruire la logica dello schema. Gli esempi seguenti usano il prefisso **vg:** e mostrano quattro tipi di conoscenza derivata.

10.1 Classi Inferite al Posto dei Filtri

```
PREFIX vg: <http://www.videogame-ontology.org/ontology#>
SELECT ?name ?score WHERE {
  ?g a vg:HighlyRatedGame , vg:MultiplayerGame , vg:PlayStationGame ;
  vg:gameName ?name ;
  vg:metacriticScore ?score .
}
ORDER BY DESC(?score) LIMIT 10
```

Listing 1: Giochi multiplayer per PlayStation con punteggio alto

Senza reasoning la stessa domanda richiederebbe un filtro sul punteggio (≥ 85), un join con le modalità di gioco e un join con le piattaforme per verificarne il tipo (ad esempio PS5 a PlayStationPlatform). Con le classi definite (Sezione 5) la query si riduce a tre asserzioni di tipo.

10.2 Relazioni Derivate da Property Chain

```
PREFIX vg: <http://www.videogame-ontology.org/ontology#>
SELECT DISTINCT ?other WHERE {
  ?er vg:gameName "Elden Ring" ;
  vg:sharesDeveloperWith ?g .
  ?g vg:gameName ?other .
  FILTER(?g != ?er)
}
```

Listing 2: Giochi dello stesso sviluppatore di Elden Ring

`sharesDeveloperWith` non è mai asserita nei dati: deriva dalla catena `developedBy` o `developerOf` (Sezione 4.3). Il `FILTER` esclude la coppia riflessiva che la catena produce per ogni gioco.

10.3 Transitività: Materializzazione o Property Path

```
PREFIX vg: <http://www.videogame-ontology.org/ontology#>
SELECT ?pred WHERE {
  ?g vg:gameName "Dark Souls III" ;
  vg:sequelOf ?pred .          # con reasoning (chiusura materializzata)
  # senza reasoning: ?g vg:sequelOf+ ?pred .
}
```

Listing 3: Tutti i predecessori di un gioco

Poiché `sequelOf` è transitiva, dopo il reasoning il grafo contiene già tutte le coppie (gioco, predecessore) a qualsiasi distanza. Lo stesso risultato si può ottenere senza reasoning con il *property path* SPARQL `vg:sequelOf+`, che calcola la chiusura al momento della query. La differenza è concettuale: nel primo caso la transitività è una proprietà del dominio dichiarata nell'ontologia e vale per ogni applicazione che la usa; nel secondo è una scelta di chi scrive la query.

10.4 Navigazione Tramite Inverse

```
PREFIX vg: <http://www.videogame-ontology.org/ontology#>
SELECT ?studio (COUNT(DISTINCT ?g) AS ?n) WHERE {
  ?dev vg:developerOf ?g ;
        vg:developerName ?studio .
  ?g a vg:AwardWinningGame .
}
GROUP BY ?studio ORDER BY DESC(?n) LIMIT 10
```

Listing 4: Studi con più giochi premiati

I dati asseriscono solo **developedBy** (dal gioco allo studio): la query parte dallo studio grazie all'inversa **developerOf** e usa la classe definita **AwardWinningGame** invece di un join esplicito con i premi.

10.5 Open World e Negazione

Una domanda come <giochi senza sequel> mette in luce la differenza tra l'ontologia e le query. In OWL vale la *Open World Assumption*: l'assenza di una tripla **sequelOf** non implica che il gioco non abbia predecessori, quindi un reasoner DL non classificherebbe nessun gioco come **StandaloneGame** senza ulteriori assiomi di chiusura. Il secondo livello di reasoning adotta invece esplicitamente una *Closed World Assumption* locale (**FILTER NOT EXISTS**, Sezione 9): **StandaloneGame** va quindi letta come <gioco per cui non è noto alcun predecessore>.

11 Popolamento dell'Ontologia

11.1 Pipeline populate_wikidata.py

La popolazione delle istanze avviene tramite lo script **populate_wikidata.py**, che interroga l'endpoint SPARQL pubblico di Wikidata. La pipeline è strutturata in quattro fasi principali.

11.1.1 Query SPARQL verso Wikidata

Vengono eseguite 12 tipologie di query SPARQL, ciascuna mirata a estrarre una specifica categoria di dati:

1. **Core** — nomi, date di rilascio, sviluppatore e publisher (P31, P577, P178, P123);
2. **Generi** — mapping genere via P136;
3. **Piattaforme** — piattaforme via P400;
4. **Personaggi** — personaggi presenti nel gioco via P674;
5. **Franchise** — serie di appartenenza via P179;
6. **Game Mode** — modalità di gioco via P404;
7. **Metacritic** — punteggio via P444 con qualificatore P447 (Metacritic);
8. **Engine** — motore grafico via P408;
9. **Paese** — paese d'origine via P495;
10. **Sito web** — sito ufficiale via P856;
11. **Premi** — premi ricevuti via P166;
12. **Descrizioni** — descrizioni testuali via **schema:description**.

Per evitare i timeout dell'endpoint pubblico Wikidata (limite ~60 secondi), le query sono partizionate per **anno × mese**: 17 anni (2010–2026) × 12 mesi = **204 iterazioni** per ciascuna tipologia di query. Tra ogni richiesta viene inserito un **polite delay** di 1 secondo. Il tempo totale di esecuzione è di circa **5 ore**.

11.1.2 Deduplicazione

Wikidata può restituire la stessa entità con URI diversi ma stesso nome normalizzato (es. entità omonime restituite da query diverse). La pipeline applica una normalizzazione del nome (trim degli spazi e lowercase) e rimuove **4.420 entità duplicate**, preservando per ciascun nome l'URI con il maggior numero di triple associate.

11.1.3 OWL-RL Reasoning

Dopo la popolazione iniziale ($\sim 1.019.589$ triple grezze), viene applicata la chiusura deduttiva OWL 2 RL tramite `owlrl`:

- Materializzazione proprietà inverse: per ogni `developedBy`, viene aggiunto `developerOf`;
- Catene `subPropertyOf`: `gameName` $\rightarrow$ `name`;
- Inferenze da classi definite (`AwardWinningGame`, `FranchiseGame`).

Il grafo cresce da $\sim 1\text{M}$ a $\sim 1.4\text{M}$ triple.

11.1.4 Pruning

Lo step finale di pruning (`prune_owl.py`) rimuove triple non necessarie a runtime:

- `owl:sameAs` riflessivi (la chiusura OWL 2 RL genera un `sameAs` di ogni entità con sé stessa);
- `gameDescription`: testi lunghi non usati nelle query;
- `officialWebsite`: URI non necessari per la ricerca semantica.

Complessivamente vengono rimosse $\sim 253\text{k}$ triple. Risultato finale: $\sim 1.173.959$ triple (dataset completo 2010–2026).

11.2 Demo Subset 2020–2026

Per rispettare i limiti di RAM del deploy su Render (512 MB), la demo live utilizza un sottoinsieme dell'ontologia limitato ai giochi dal 2020 in poi (`prune_years.py`):

- ~ 745.400 triple;
- ~ 68.700 giochi;
- RAM a runtime: ~ 250 MB (`pyoxigraph`).

11.3 Performance: `rdflib` vs `pyoxigraph`

La Tabella 13 confronta le performance dei due store RDF utilizzati nel progetto. Il passaggio da `rdflib` (Python puro) a `pyoxigraph` (binding Rust) ha prodotto un miglioramento drastico.

Tabella 13: Confronto performance `rdflib` vs `pyoxigraph`

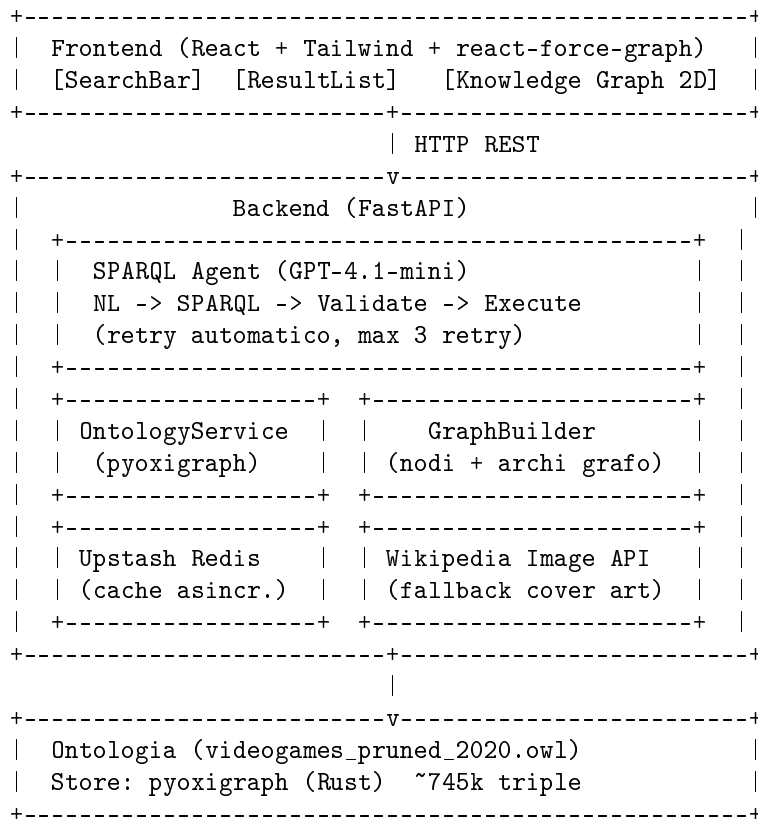
Metrica	<code>rdflib</code>	<code>pyoxigraph</code> (completo)	<code>pyoxigraph</code> (demo)
RAM a runtime	~ 1.550 MB	~ 389 MB	~ 250 MB
Tempo di caricamento	$\sim 41\text{s}$	$\sim 2.2\text{s}$	$\sim 1.5\text{s}$
Query SPARQL tipica	1–23s	0.08–0.37s	0.05–0.25s
Speedup query	—	5–62 $\times$	—

`pyoxigraph` [Tpt and contributors, 2024] garantisce un caricamento quasi istantaneo (< 3 secondi contro 41 secondi) e query 5–62 volte più veloci, rendendo l'esperienza utente fluida anche su hardware limitato.

12 Architettura del Sistema

12.1 Diagramma Architeturale

L'architettura del sistema segue un classico pattern a tre livelli (three-tier): frontend di presentazione, backend API REST e store ontologico. Il diagramma seguente illustra il flusso dati:



12.2 Stack Tecnologico

La Tabella 14 riassume le tecnologie impiegate per ciascun livello.

Tabella 14: Stack tecnologico del sistema

Layer	Tecnologia
Frontend	React 18, TypeScript, Vite, Tailwind CSS
Grafo interattivo	react-force-graph-2d
Backend	FastAPI (Python 3.12+, async)
Ontologia (runtime)	pyoxigraph (Rust), OWL 2
Ontologia (offline)	rdflib + owlrl (reasoning, validazione)
LLM / Agente	OpenAI GPT-4.1-mini
Cache	Upstash Redis (async) + cache in-memory
Fonte dati	Wikidata SPARQL
Deploy	Render (backend) + Vercel (frontend)

12.3 Flusso Dati

Il flusso di una tipica interrogazione è il seguente:

1. L'utente digita una domanda in linguaggio naturale (NL) nella SearchBar del frontend;
2. Il frontend invia la domanda al backend via HTTP POST su `/api/query`;
3. Lo **SPARQL Agent** (GPT-4.1-mini) genera una query SPARQL a partire dalla domanda NL, utilizzando il contesto dell'ontologia (classi, proprietà, esempi);
4. Il **SPARQL Validator** controlla con rdflib la correttezza sintattica della query generata; se la query è invalida, fallisce in esecuzione o restituisce zero risultati, l'agente ritenta (fino a 3 retry, quindi al massimo 4 tentativi);
5. La query viene eseguita su **pyoxigraph** contro il file OWL;
6. Il **GraphBuilder** trasforma i risultati SPARQL in nodi e archi per il frontend (con metadati di tipo, label, colore);
7. Il frontend riceve i risultati e li visualizza nella ResultList e nel Knowledge Graph 2D interattivo.

12.4 Cache a Due Livelli

Per ottimizzare le performance, il sistema adotta una cache a due livelli:

- **Upstash Redis** (cloud, persistente): memorizza i risultati delle query (TTL 14 giorni) e le immagini (cover art) recuperate da Wikipedia (TTL 6 mesi). I risultati vuoti non vengono cachati per consentire retry successivi;
- **Cache in-memory** (Python dict): mantiene in RAM label e tipi dei nodi, popolati al primo accesso, e funge da fallback per risultati e immagini quando Redis non è disponibile.

12.5 Tabella dei Componenti

La Tabella 15 elenca i principali componenti software con le rispettive responsabilità.

Tabella 15: Componenti principali e responsabilità

Componente	Responsabilità
<code>sparql_agent.py</code>	Traduzione NL → SPARQL via GPT-4.1-mini, retry
<code>sparql_validator.py</code>	Validazione sintattica SPARQL pre-esecuzione
<code>ontology_service.py</code>	Caricamento OWL, esecuzione query via pyoxigraph
<code>graph_builder.py</code>	Costruzione nodi/archi da risultati RDF per il frontend
<code>image_cache.py</code>	Recupero e cache immagini da Wikipedia API
<code>SearchBar.tsx</code>	Input NL con domande suggerite
<code>ResultList.tsx</code>	Tabella dei risultati; le URI aprono il dettaglio nodo
<code>KnowledgeGraph.tsx</code>	Grafo 2D interattivo con force layout, zoom, filtri
<code>NodeDetail.tsx</code>	Pannello laterale con dettaglio nodo (proprietà, relazioni)
<code>NodeContextMenu.tsx</code>	Menu contestuale: relazioni per predicato, espandi, nascondi nodo
<code>SparqlViewer.tsx</code>	Visualizzazione query SPARQL generata (espandibile)

12.6 API Endpoint

La Tabella 16 riassume gli endpoint REST esposti dal backend.

13 Frontend e Backend

13.1 Frontend: React + TypeScript + Vite

L'interfaccia utente è un'applicazione single-page costruita con **React 18**, **TypeScript**, **Vite** e **Tailwind CSS** [Meta Platforms, 2024]. Il componente centrale è il **Knowledge Graph 2D**,

Tabella 16: Endpoint API

Metodo	Endpoint	Descrizione
POST	/api/query	Ricerca NL → risultati + grafo
POST	/api/cross-links	Archivi tra nodi già presenti e nuovi nodi
GET	/api/node/{uri}	Dettaglio nodo + sottografo
GET	/api/image-search	Cerca cover art da Wikipedia
GET	/api/stats	Statistiche ontologia
GET	/api/health	Health check

realizzato con la libreria **react-force-graph-2d**, che visualizza i risultati delle query come un grafo interattivo physics-based.

13.1.1 Componenti Principali

I sei componenti React che compongono l'interfaccia sono:

- **SearchBar** — input di ricerca in linguaggio naturale, con un elenco di domande suggerite mostrato al focus;
- **ResultList** — tabella dei risultati della query, con una colonna per ogni variabile SPARQL. I valori URI sono cliccabili: aprono il dettaglio del nodo e centrano il grafo su di esso;
- **KnowledgeGraph** — il grafo interattivo 2D con force-directed layout. Supporta zoom, pan, hover (alone sul nodo ed etichette degli archi a zoom elevato) e un pulsante “recentra” per zoom-to-fit;
- **NodeDetail** — pannello laterale che mostra tutte le proprietà e relazioni del nodo selezionato. Include link cliccabili per espandere altri nodi nel grafo;
- **NodeContextMenu** — menu contestuale al click destro (o long-press) su un nodo: relazioni raggruppate per predicato e direzione, espansione di un singolo vicino, di un gruppo o di tutti, apertura nella sidebar e rimozione (nascondimento) del nodo dal grafo;
- **SparqlViewer** — pannello espandibile che mostra la query SPARQL generata dall'agente, utile per debugging e trasparenza.

13.1.2 Caratteristiche del Grafo Interattivo

Il Knowledge Graph offre un'esperienza di navigazione ricca:

- **Nodi colorati per tipo:** VideoGame (indaco), Developer (ambra), Publisher (verde), Genre (rosso), Platform (blu), Franchise (viola), Award (arancione), ecc.;
- **VideoGame come rettangoli portrait:** i nodi gioco sono visualizzati come riquadri verticali con immagine di copertina (da Wikipedia) o emoji placeholder, differenziandosi dagli altri nodi circolari;
- **Animazione physics-based:** forza di attrazione/repulsione tra i nodi con smorzamento progressivo;
- **Long-press su mobile:** supporto touch per dispositivi mobili;
- **Filtro per tipo:** legenda interattiva che permette di nascondere/mostrare intere categorie di nodi con un click;
- **Selezione:** il nodo cliccato viene evidenziato e portato in primo piano (z-order);
- **Cross-link:** dopo ogni query o espansione il backend (/api/cross-links) individua gli archi tra i nodi già presenti e quelli nuovi, mantenendo il grafo connesso.

13.2 Backend: FastAPI Async

Il backend è sviluppato con **FastAPI** [Ramírez and contributors, 2024] in modalità asincrona, esponendo 6 endpoint REST (cfr. Tabella 16, Sezione 12). I componenti chiave sono:

13.2.1 SPARQL Agent

Il cuore intelligente del sistema è lo `sparql_agent.py`, che utilizza **GPT-4.1-mini** di OpenAI [OpenAI, 2025] per tradurre domande in linguaggio naturale (italiano o inglese) in query SPARQL. Il prompt include:

- Un riassunto dello schema dell'ontologia (classi principali, classi definite, proprietà e catene rilevanti per l'interrogazione);
- I prefissi SPARQL (`vg:`, `rdf:`, `xsd:`);
- Esempi di domande e query corrispondenti (*few-shot prompting*);
- Istruzioni per generare solo la query SPARQL, senza testo aggiuntivo.

Il flusso include un meccanismo di **retry automatico** (fino a 3 retry, cioè al massimo 4 tentativi):

1. L'agente genera una query SPARQL;
2. Il `sparql_validator` (rdflib) ne verifica la correttezza sintattica; la query viene poi eseguita su pyoxigraph;
3. Se la query è sintatticamente invalida, fallisce in esecuzione o produce zero risultati, l'agente riceve il messaggio di errore e le query fallite precedenti, e ritenta con un prompt correttivo.

13.2.2 GraphBuilder

Il `graph_builder.py` trasforma i risultati SPARQL (binding di variabili) in una struttura JSON adatta al frontend:

- **Nodi**: URI, label, tipo (classe OWL), colore e dimensione;
- **Archi**: sorgente, destinazione e label (nome locale del predicato).

La forma dei nodi è decisa dal frontend in base al tipo, e le immagini di copertina sono richieste separatamente tramite `/api/image-search`.

Il builder arricchisce i nodi con metadati derivati dal grafo RDF (tipo tramite `rdf:type`, label tramite le proprietà `*Name`).

13.2.3 Cache a Due Livelli

- **Upstash Redis** [Inc., 2025]: cache cloud persistente per i risultati delle query (TTL 14 giorni) e gli URL delle immagini (TTL 6 mesi);
- **Cache in-memory**: dizionari Python per label e tipi, popolati al primo accesso e consultabili in $O(1)$; fungono anche da fallback per risultati e immagini quando Redis non è disponibile.

13.3 Endpoint API

Si rimanda alla Tabella 16 nella Sezione 12 per l'elenco completo degli endpoint REST.

14 Conclusione

14.1 Riepilogo dei Risultati

Il progetto **Videogame Semantic Search** ha prodotto un sistema completo di ricerca semantica che integra:

- **Un'ontologia OWL 2 DL** espressiva e ben strutturata, evoluta da 11 a 78 classi (61 primitive, 15 definite e 2 con vincolo di cardinalità), con 31 object property, 26 datatype property, 12 coppie inverse, 3 property chain, 3 proprietà transitive, 12 blocchi di disjointness e allineamento esterno a Wikidata e Dublin Core, nel rispetto dei vincoli globali di OWL 2 DL sulle proprietà;

- **Un sistema di reasoning a due livelli** (OWL 2 RL + 21 regole CONSTRUCT) funzionante end-to-end, con 0 falsi `owl:sameAs` dopo il fix del bug critico degli assiomi DL-only;
- **Un knowledge graph popolato** da Wikidata con oltre 1.1 milioni di triple (dataset completo 2010–2026) e ~745k triple nel sottoinsieme demo (2020–2026, ~68.700 giochi);
- **Un’interfaccia web interattiva** con ricerca in linguaggio naturale, agente SPARQL intelligente, grafo 2D navigabile e filtri per tipo;
- **Performance ottimizzate** grazie al passaggio da rdflib a pyoxigraph (query 5–62× più veloci, caricamento in <3s, RAM ~250 MB sulla demo).

14.2 Possibili Estensioni Future

Il sistema offre numerose direzioni di sviluppo futuro:

1. **Reasoner DL completo** — integrare HermiT o Pellet tramite `owlready2` per ragionare sui costrutti esterni al profilo RL (datatype restriction, complementOf) e sugli assiomi DL-only (FunctionalProperty, hasKey) senza bisogno delle regole CONSTRUCT compensative, previa migrazione delle date a `xsd:dateTime` (cfr. Sezione 8). Questo abiliterebbe anche la rilevazione automatica di inconsistenze tra le istanze;
2. **Validazione delle istanze con SHACL** — definire SHACL shapes per validare la conformità delle istanze (es. ogni VideoGame deve avere almeno un `gameName`, le date devono essere nel formato corretto);
3. **SWRL rules reali** — sostituire le 21 regole CONSTRUCT con autentiche SWRL rules, sfruttando un reasoner che le supporti nativamente;
4. **Estensione della pipeline Wikidata** — aggiungere query per recuperare `hasSteamAppId`, `budgetTier`, `playerCount` e la relazione `sequelOf`, attualmente non popolate dai dati Wikidata;
5. **Internazionalizzazione del frontend** — supporto multilingua per l’interfaccia (oggi in italiano) ed estensione delle query in linguaggio naturale oltre italiano e inglese (francese, spagnolo, ...);
6. **Visualizzazione avanzata delle classi definite** — nel grafo interattivo, evidenziare con badge o colori speciali i nodi che sono stati classificati automaticamente dal reasoner (es. bordino dorato per `HighlyRatedGame`, icona controller per `MultiplayerGame`);
7. **SPARQL endpoint pubblico** — esporre un endpoint SPARQL standard (RESTful o tramite Triplestore) per consentire query federate da applicazioni esterne.

Il progetto dimostra come le tecnologie del Semantic Web — ontologie formali, reasoning automatico, knowledge graph e query SPARQL — possano essere integrate in un’applicazione web moderna e performante, offrendo un’esperienza di ricerca semantica che va ben oltre la semplice ricerca per parole chiave.

A Esempio di Istanze di Test

Per la verifica end-to-end del sistema di reasoning, è stato utilizzato un insieme di istanze di test minimale (`test_abox.ttl`) contenente 3 giochi, 2 developer e alcune piattaforme. Il seguente estratto in Turtle illustra la struttura:

```
@prefix vg: <http://www.videogame-ontology.org/ontology#> .

vg:GameA a vg:VideoGame ;
  vg:gameName "Elden Ring" ;
  vg:releaseDate "2022-02-25"^^xsd:date ;
  vg:metacriticScore 96 ;
  vg:developedBy vg:Dev1 ;
  vg:hasGenre vg:RPGGenreInstance ;
```

```

    vg:hasGameMode vg:SinglePlayer, vg:OnlineMultiplayer ;
    vg:availableOn vg:PS5, vg:PC_Windows ;
    vg:wonAward vg:TGAAward2022 .

vg:GameB a vg:VideoGame ;
    vg:gameName "Hollow Knight" ;
    vg:releaseDate "2017-02-24"^^xsd:date ;
    vg:metacriticScore 87 ;
    vg:developedBy vg:Dev2 ;
    vg:availableOn vg:NintendoSwitch ;
    vg:hasGameMode vg:SinglePlayer .

vg:GameC a vg:VideoGame ;
    vg:gameName "Dark Souls III" ;
    vg:sequelOf vg:GameX ;
    vg:developedBy vg:Dev1 .

# individui tipizzati (necessari per le restrizioni esistenziali)
vg:PS5 a vg:PlayStationPlatform .
vg:PC_Windows a vg:WindowsPlatform .
vg:NintendoSwitch a vg:NintendoPlatform .
vg:SinglePlayer a vg:SinglePlayerMode .
vg:OnlineMultiplayer a vg:MultiplayerMode .
vg:TGAAward2022 a vg:IndustryAward .

```

Listing 5: Estratto del test delle istanze (Turtle)

A.1 Risultati della Verifica

Dopo il reasoning completo (owlrl + CONSTRUCT), il grafo di test produce:

- **2.715 triple finali** (dalle ~997 iniziali);
- **0 owl:sameAs falsi** — il fix degli assiomi DL-only previene l'unificazione errata dei giochi;
- **Classificazioni corrette:**
 - GameA → MultiplayerGame, SinglePlayerGame, ConsoleGame, PlayStationGame, PC-Game, HighlyRatedGame, RecentGame, AwardWinningGame, StandaloneGame;
 - GameB → SinglePlayerGame, ConsoleGame, NintendoGame, HighlyRatedGame, StandaloneGame (non RecentGame: 2017);
 - GameC → SequelGame (nessuna classificazione per piattaforma o modalità: non sono asseriti availableOn né hasGameMode).
- **Proprietà derivate:**
 - Dev1 developerOf GameA, GameC (per l'inversa di developedBy);
 - GameA sharesDeveloperWith GameC e viceversa (catena developedBy o developerOf e simmetria);
 - GameX hasSequel GameC (inversa); GameX è inoltre classificato VideoGame per effetto del range di sequelOf.

B Sintassi DL delle Classi Definite

Di seguito si riporta la sintassi DL compatta delle 15 classi definite dell'ontologia e delle 2 classi con vincolo di cardinalità. La notazione $\sqcap$ indica intersezione (`owl:intersectionOf`), $\exists$ indica restrizione esistenziale (`owl:someValuesFrom`), $\neg$ indica complemento (`owl:complementOf`), $\leq$ indica cardinalità massima (`owl:maxCardinality`).

$$\begin{aligned}
\text{MultiplayerGame} &\equiv \text{VideoGame} \sqcap \exists \text{hasGameMode}.\text{MultiplayerMode} \\
\text{SinglePlayerGame} &\equiv \text{VideoGame} \sqcap \exists \text{hasGameMode}.\text{SinglePlayerMode} \\
\text{CoopGame} &\equiv \text{VideoGame} \sqcap \exists \text{hasGameMode}.\text{CoopMode} \\
\text{PCGame} &\equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{PCPlatform} \\
\text{ConsoleGame} &\equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{ConsolePlatform} \\
\text{MobileGame} &\equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{MobilePlatform} \\
\text{NintendoGame} &\equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{NintendoPlatform} \\
\text{PlayStationGame} &\equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{PlayStationPlatform} \\
\text{XboxGame} &\equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{XboxPlatform} \\
\text{SequelGame} &\equiv \text{VideoGame} \sqcap \exists \text{sequelOf}.\text{VideoGame} \\
\text{StandaloneGame} &\equiv \text{VideoGame} \sqcap \neg \exists \text{sequelOf}.\text{VideoGame} \\
\text{HighlyRatedGame} &\equiv \text{VideoGame} \sqcap \exists \text{metacriticScore}.\text{integer}[\geq 85] \\
\text{RecentGame} &\equiv \text{VideoGame} \sqcap \exists \text{releaseDate}.\text{date}[\geq 2020-01-01] \\
\text{AwardWinningGame} &\equiv \text{VideoGame} \sqcap \exists \text{wonAward}.\text{Award} \\
\text{FranchiseGame} &\equiv \text{VideoGame} \sqcap \exists \text{belongsTo}.\text{Franchise} \\
\text{GameWithSingleDeveloper} &\sqsubseteq \text{VideoGame} \sqcap \leq 1 \text{ developedBy} \\
\text{ExclusiveRelease} &\sqsubseteq \text{VideoGame} \sqcap \leq 1 \text{ availableOn}
\end{aligned}$$

Le prime 15 classi utilizzano $\equiv$ (condizioni necessarie e sufficienti, `owl:equivalentClass`); le ultime due usano $\sqsubseteq$ (sole condizioni necessarie, `rdfs:subClassOf`).

C Assiomi dello Schema in Turtle

Lo schema è memorizzato in RDF/XML (`videogames.owl`). Gli estratti seguenti lo riportano in Turtle, più leggibile, per mostrare come la sintassi DL delle sezioni precedenti diventa un insieme di triple RDF. Restrizioni, intersezioni e liste sono rappresentate da nodi blank (`[ ... ]` e `( ... )`). Si omettono `rdfs:label` e `rdfs:comment`.

Classe definita con restrizione esistenziale

```

vg:PlayStationGame a owl:Class ;
  rdfs:subClassOf vg:VideoGame ;
  owl:equivalentClass [ a owl:Class ;
    owl:intersectionOf ( vg:VideoGame
      [ a owl:Restriction ;
        owl:onProperty vg:availableOn ;
        owl:someValuesFrom vg:PlayStationPlatform ] ) ] .

```

$\text{PlayStationGame} \equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{PlayStationPlatform}$. L'intersezione è una lista RDF di due elementi: la classe `VideoGame` e una restrizione anonima.

Classe definita con datatype restriction

```

vg:HighlyRatedGame a owl:Class ;
  rdfs:subClassOf vg:VideoGame ;
  owl:equivalentClass [ a owl:Class ;
    owl:intersectionOf ( vg:VideoGame
      [ a owl:Restriction ;
        owl:onProperty vg:metacriticScore ;
        owl:someValuesFrom [ a rdfs:Datatype ;
          owl:onDatatype xsd:integer ;

```

```
owl:withRestrictions ( [ xsd:minInclusive 85 ] ) ] ] )
] .
```

Il filler di `someValuesFrom` non è una classe ma un *data range*: il sottoinsieme di `xsd:integer` vincolato dal facet `xsd:minInclusive 85`.

Classe definita con complemento

```
vg:StandaloneGame a owl:Class ;
  rdfs:subClassOf vg:VideoGame ;
  owl:equivalentClass [ a owl:Class ;
    owl:intersectionOf ( vg:VideoGame
      [ a owl:Class ;
        owl:complementOf [ a owl:Restriction ;
          owl:onProperty vg:sequelOf ;
          owl:someValuesFrom vg:VideoGame ] ] ) ] ] .
```

$\text{StandaloneGame} \equiv \text{VideoGame} \sqcap \neg \exists \text{sequelOf}.\text{VideoGame}$.

Condizione solo necessaria con cardinalità

```
vg:GameWithSingleDeveloper a owl:Class ;
  rdfs:subClassOf vg:VideoGame ,
  [ a owl:Restriction ;
    owl:onProperty vg:developedBy ;
    owl:maxCardinality "1"^^xsd:nonNegativeInteger ] .
```

Due assiomi `rdfs:subClassOf` distinti, nessun `owl:equivalentClass`:

$\text{GameWithSingleDeveloper} \sqsubseteq \text{VideoGame} \sqcap \leq 1 \text{developedBy}$

L'assenza di `owl:onClass` rende la cardinalità non qualificata.

Proprietà transitiva e inversa

```
vg:sequelOf a owl:ObjectProperty , owl:TransitiveProperty ;
  rdfs:domain vg:VideoGame ;
  rdfs:range vg:VideoGame ;
  owl:inverseOf vg:hasSequel .

vg:hasSequel a owl:ObjectProperty ;
  rdfs:domain vg:VideoGame ;
  rdfs:range vg:VideoGame .
```

Property chain

```
vg:sharesDeveloperWith a owl:ObjectProperty , owl:SymmetricProperty ;
  rdfs:domain vg:VideoGame ;
  rdfs:range vg:VideoGame ;
  owl:propertyChainAxiom ( vg:developedBy vg:developerOf ) .
```

La catena è una lista ordinata: l'ordine delle proprietà conta (`developedBy` $\circ$ `developerOf`).

Disjointness e chiavi

```
[[] a owl:AllDisjointClasses ;  
    owl:members ( vg:ConsolePlatform vg:PCPlatform vg:MobilePlatform ) .  
  
vg:Review owl:hasKey ( vg:reviewedGame vg:reviewerName ) .
```

`owl:AllDisjointClasses` dichiara disgiunte a coppie tutte le classi della lista con un solo assioma, invece di $n(n-1)/2$ assiomi `owl:disjointWith`.

Bibliografia

- Edmond Chuc and contributors. owlrl: OWL-RL reasoning for RDFLib, 2024. URL <https://github.com/RDFLib/owlrl>.
- Wikimedia Foundation. Wikidata: The Free Knowledge Base, 2024. URL <https://www.wikidata.org/>.
- Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993. doi: 10.1006/knac.1993.1008.
- Pascal Hitzler, Markus Krötzsch, Bijan Parsia, Peter F. Patel-Schneider, and Sebastian Rudolph. OWL 2 Web Ontology Language Primer (Second Edition). Technical report, W3C, 2012. URL <https://www.w3.org/TR/owl2-primer/>.
- Upstash Inc. Upstash Redis, 2025. URL <https://upstash.com/>.
- Dublin Core Metadata Initiative. DCMI Metadata Terms. Technical report, DCMI, 2020. URL <https://www.dublincore.org/specifications/dublin-core/dcmi-terms/>.
- Inc. Meta Platforms. React: A JavaScript library for building user interfaces, 2024. URL <https://react.dev/>.
- Boris Motik, Bernardo Cuenca Grau, Ian Horrocks, Zhe Wu, Achille Fokoue, and Carsten Lutz. OWL 2 Web Ontology Language Profiles (Second Edition). Technical report, W3C, 2012. URL <https://www.w3.org/TR/owl2-profiles/>.
- OpenAI. GPT-4.1 mini: OpenAI API, 2025. URL <https://platform.openai.com/>.
- Sebastián Ramírez and contributors. FastAPI: modern web framework for building APIs, 2024. URL <https://fastapi.tiangolo.com/>.
- Tpt and contributors. pyoxigraph: Python bindings for Oxigraph, 2024. URL <https://github.com/oxigraph/oxigraph>.